

Embeddings Models - BERT and GPT

Naresh Chaurasia

Version 1.0, 2026-07-03: First Edition

Table of Contents

1. Bert V/s GPT	1
-----------------------	---

Chapter 1. Bert V/s GPT

BERT = Bidirectional Encoder Representations from Transformers

GPT = Generative Pre-trained Transformer

Both names actually map directly onto what you've already covered:

- **BERT** — "Bidirectional" and "Encoder" point to exactly the mechanism you just taught: bidirectional attention, encoder-only architecture.
- **GPT** — "Generative" and "Pre-trained Transformer" point to its purpose: generating text, using a Transformer that's pre-trained (originally on next-token prediction) before being fine-tuned or used directly.

BERT vs GPT — how they read text

BERT-based models use bidirectional attention — when processing a word, the model looks at *all* the other words in the sentence at once, the ones before it AND the ones after it, to build that word's representation. It reads the entire sentence, but the key point isn't just "it sees everything" — it's *how* each word's meaning gets built using context from both directions simultaneously.

GPT-based models use unidirectional (causal) attention. GPT also technically "sees" the whole input you give it, but when computing the representation for a given word/token, it's only allowed to look at words *before* it, not after. This is enforced by a "causal mask" in the attention mechanism — a hard restriction, not just a training habit.

A concrete example to make it click

Take the word "bank" in: *"I sat on the river bank to relax."*

- **BERT (bidirectional)**: When building the vector for "bank", it looks at "river" (before) AND "to relax" (after). Seeing "river" *and* the relaxing context together makes it very confident this means the *land beside a river*, not a financial institution.
- **GPT (unidirectional)**: When it reaches "bank", it has only seen "I sat on the river" so far — it hasn't seen "to relax" yet, because those words come after. It still gets a decent signal from "river", but it's making its prediction with less information than BERT has at that same point.



Bidirectional = each word's vector is built using context from **both** the words before it and after it, at the same time — not just "the model eventually reads the whole sentence."

Putting it to work: BERT for embedding, GPT for generation

The bank example shows *how* each model reads. These next two examples show *what each model is used for* — using a ShopSmart India support ticket.

BERT-based example (understanding/embedding task)

Input ticket: *"My order hasn't arrived and I'm not able to track it."*

BERT reads the whole sentence bidirectionally and outputs a single vector like:

```
[0.021, -0.034, 0.198, ..., 0.077] # 384 dimensions for MiniLM
```

This vector captures the *meaning*—"delivery delay + tracking issue"—so FAISS can find similar past tickets (e.g., "Package delayed, tracking link broken") even if the wording is completely different. That's the whole point of the NB-01 pipeline: BERT-family models don't generate anything, they just produce a good numerical fingerprint of meaning, using the same bidirectional context described above.

GPT-based example (generation task)

Input prompt: *"Write a reply to a customer whose ShopSmart India order is delayed."*

GPT reads left-to-right and generates the next tokens one at a time, using only the causal, one-directional context described above:

```
"We're" -> "sorry" -> "for" -> "the" -> "delay" -> "in" -> "your" -> "order" -> ...
```

Each word is predicted only from the words *before* it—it never peeks ahead—and it keeps going until it produces a full, coherent customer support reply.



BERT turns a whole sentence into **one vector** (used for search/matching). GPT turns a prompt into a **sequence of new words** (used for writing/responding). Both behaviors trace directly back to the same root cause: whether attention looks in one direction or both.